Method

The sequential reference, pthread implementation, and OpenMP implementation were compiled with GCC using -O2. Each variant ran against both supplied JPEG images with 1, 2, 3, 4, and 8 requested threads for the parallel versions. The times beloware medians of three executions, reported in seconds. The sequential and pthread programs use CLOCK_MONOTONIC. OpenMP uses omp_get_wtime. Image loading and BMP output are not timed. The sequential timer covers the Sobel convolution. Pthread timing includes creating and joining its workers, while OpenMP timing includes entering and leaving the parallel region, including its completion barrier.The different threading models therefore carry their own setup costs in the reported parallel measurements.

Result

NGGYU.

Method	1	2	3	4	8
Sequential	0.012429				
Pthreads	0.018951	0.008091	0.008187	0.004976	0.002954
OpenMP	0.011221	0.005899	0.006838	0.006637	0.004792

NGGYU2

Method	1	2	3	4	8
Sequential	0.066740				
Pthreads	0.083436	0.044734	0.036924	0.034750	0.032738
OpenMP	0.096526	0. 043351	0. 045607	0. 035540	0. 053810

Observations

The larger image took longer to process sequentially than the smaller image because it contained more pixels to process. For the large image, the best pthread result was a median of 0.032738 seconds using eight threads, compared with 0.066740 seconds for sequential execution. The two-thread pthread and OpenMP implementations were also faster than the sequential version, showing that dividing the pixel-processing work among multiple threads can improve performance.

However, increasing the number of threads did not always improve performance. For the large image, the four-thread OpenMP version achieved a median of 0.035540 seconds, while the eight-thread version took 0.053810 seconds. This suggests that adding more threads can introduce overhead from scheduling and synchronization, as well as competition for CPU resources and memory bandwidth. Since each output pixel requires reading a 3-by-3 neighborhood, memory access also plays an important role in performance.

For NGGYU.jpg, the workload was relatively small, so thread creation and management overhead had a greater effect on execution time. For example, using one pthread worker was slower than sequential execution because creating and joining a thread adds extra work. The results for NGGYU2.jpg also showed that the one-thread pthread and OpenMP implementations could be slower than sequential execution.

Overall, these results show that parallel processing can improve performance, but the benefits depend on the image size and number of threads used. Actual timings can also vary depending on CPU availability, background processes, compiler versions, and system scheduling. Since each configuration was tested only three times, small differences between results should be interpreted cautiously. To obtain more reliable results, I would increase the number of repetitions, report the variation in execution times, and perform the tests on an otherwise idle machine.